
KCU105 Complete User Manual

GitHub Repository: https://github.com/ANDnwl-IITB/Board_git_files_NWL

1. File Locations

The repository already contains the complete KCU105 folder hierarchy. Navigate to:

`Board_git_files_NWL/SBC/kcu105/`

Important folders:

- **TESTS** – DRAM and Ethernet tests.
- **constraint** – Board constraint files (.xdc).
- **edn** – Processor .edn file/link.
- **hsys** – AA synthesis connections and components.
- **hsys__dram** – Processor, DRAM and UART connections.
- **hsys__processor__sram** – SRAM block and core.
- **Ip** – Required IPs.
- **Toplevel** – Top-level RTL and port mapping.
- **Verilog** – Verilog files.
- **vhdl** – VHDL files.
- **vivado__synth** – Bitstream-generation TCL scripts.
- **vhdl__libs** – Required VHDL libraries.

2. Bitstream Generation Steps (KCU105)

2.1. SSH into the madhav system

SSH into the madhav system (the server system).

2.2. Git clone the repository

Git clone the repository only if it is not already present on the desktop.

2.3. Enter the EDN directory

Enter the directory:

`Board_git_files_NWL/SBC/kcu105/edn/`

2.4. Set up the processor EDN link

The .edn file is needed for bitstream generation, so follow this strictly:

1. Set the path to the processor .edn file.
2. Run `./link.sh` to set up the EDN link.

If any change is made to the .edn file, first run `./build.sh` inside the **hsys** folder (in the **ajit-toolchain docker**, since it contains the Ahir-related libraries).

Then set the path to the .edn file from its actual location inside the madhav system:

`/Ajit_ddr/AjitHpcBoards/edn/ultrascale7/processor_1x1x32.edn`

2.5. Check the bitstream-generation script

Once the EDN is soft-linked, enter the directory:

```
.../kcu105/vivado_synth
```

Check the `bitgen.tcl` file.

Note:

`bitgen_dram.tcl` is the script for running the SBC (**Processor + DRAM**) design, and `bitgen.tcl` is the script for running the SBC (**Processor + DRAM + Ethernet**) design.

We use the `bitgen.tcl` script.

2.6. Edit the bitgen.tcl file

Edit the `bitgen.tcl` file, i.e., set the path correctly for all the libraries and the EDN, then proceed to the next step.

2.7. Run the synthesis script

Run the following command inside `vivado_synth`:

```
./run.sh
```

2.8. Generated files

The bitstream will be generated as `.bit` and `.ltx` files.

Note: A bitfile is already available inside the **KCU105_bitfiles** folder.

If any change is made to the EDN, follow Step 2.4 onwards.

2.9. Store the generated files

Create a new folder inside:

```
Board_git_files_NWL/KCU105_bitfiles/
```

to store the newly generated `.bit` and `.ltx` files.

Alternatively, keep the files in the same folder by removing the older one.

3. How to Use the Board – KCU105

3.1. Copy the generated files

Copy the newly generated `.bit` and `.ltx` files from:

```
Board_git_files_NWL/SBC/kcu105/vivado_synth/
```

and the **KCU105_bitfiles/** directory. These are now ready for dumping into the FPGA.

3.2. Connect the interfaces

Connect the following to the CPU:

- Serial UART module (from `.xdc`)
- Debug UART
- JTAG

3.3. Power on the board

1. Power on the board.
2. Open Vivado.
3. Connect the device.

3.4. Finish the test

Once the test completes:

1. Power off the board's switch.
2. Remove the connections from the CPU.

4. How to Run Tests on KCU105

4.1. Common Steps for All Tests

4.1.1. Go to the ajit build directory

Go to:

```
ajit-toolchain/docker/ajit_build_dev
```

4.1.2. Identify the UART devices

Type the following command and press Tab:

```
export AJIT_DEBUG_UART=/dev/ttyUSB
```

You will see options such as : `ttyUSB0` , `ttyUSB0` , `ttyUSB0` ...

Remove the UART – one of the above `ttyUSB` entries will disappear; that is the serial UART.

The other, higher-indexed entry will be the debug UART.

Example: serial UART is `ttyUSB2`, debug UART is `ttyUSB1`.

4.1.3. Set the debug UART

Run : `export AJIT_DEBUG_UART=/dev/ttyUSB1`

4.1.4. Set the serial UART

Run : `export AJIT_SERIAL_UART=/dev/ttyUSB2`

4.1.5. Run the UART setup

Run : `./run_with_uarts.sh` (you should see 0 errors)

4.1.6. Attach the shell

Run : `./attach_shell.sh`

4.2. Set the AJIT environment

Inside the `ajit-toolchain` directory, run:

```
source set_ajit_home
source ajit_env
```

4.3. Go to the KCU105 tests directory

Go to:

```
ajit-toolchain/Board_git_files_NWL/SBC/kcu105/TESTS/
```

4.4. Program the FPGA

Open Vivado and perform the following:

1. Click on **Hardware Manager**.
2. Select **Auto Connect**.
3. Select **Program**.
4. Load the bit file.

5. Print HelloWorld

5.1. Reset Sequence

- Toggle RESET_TO_PROCESSOR: 0 -> 1 -> 0
- Toggle RESET_TO_NIC: 0 -> 1 -> 0

5.2. Run the debug monitor

```
Run:  calibrateUart /dev/ttyUSB1 1000000 && ajit_debug_monitor_mt -u /dev/ttyUSB1  
-n 1 -t 1 -B 1000000 -0 (give Debug Uart number for ttyUSB)
```

5.3. Check the processor mode

```
Run:  r mode
```

You should see: `r mode returns 0x0` (This means the processor is in idle state)

5.4. Configure Minicom

Open Xterm and run: `sudo minicom -s`

Select **Serial Port Setup**, press **A**, and change the USB index to what you obtained earlier. Press Enter and exit.

5.5. Run the script

```
Run:  s run.script
```

This will show the initialization, and print statements will appear on the Minicom terminal.

5.6. Exit

Press **Ctrl + C** twice to exit.

6. Ethernet Tests

6.1. Reset Sequence

1. Set `rst_125mh_int_uc` to 1.
2. Program the device.
3. Toggle RESET_TO_PROCESSOR: 0 -> 1 -> 0.
4. Toggle RESET_TO_NIC: 0 -> 1 -> 0.

5. Set `rst_125mh_int_uc` to 0.

6.2. Perform the HelloWorld steps

Perform Steps 2 to 5 from the **Print HelloWorld** section above.

6.3. Open the Ethernet test directory

Open a new terminal and go to the following directory, not through docker:

```
ajit-toolchain/Board_git_files_NWL/SBC/kcu105/TESTS/Ethernet_tests/  
testing_NIC1.2/host
```

6.4. Run the throughput test

Run : `sudo python3 throughput.py`

6.5. Provide the test arguments

Provide the following arguments:

Number of packets	1000
Burst size	4
Packet length	1486

6.6. Expected result

You should see no packet loss.

Note: Other tests are also available – for example, the Ping test, Reliable Image Sender test, etc.

7. Marches Tests

Follow the same steps as the **Print HelloWorld** section (Section 5).